Investigating NYC Property Sales

Tien Tyler Le

May 15, 2025

Table of Contents

Data Loading & Merching

Reads in the five borough rolling-sales CSVs and the annual CPI file into R. Rolling sales were obtained from the New York City Department of Finance Rolling Sales Data.

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE

##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union

## Reading layer `MapPLUTO' from data source
##   `/Users/tienle/Downloads/nycdata/nyc_mappluto_25v1_1_shp/MapPLUTO.shp'
##   using driver `ESRI Shapefile'
## Simple feature collection with 856734 features and 94 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: 913128.9 ymin: 120049 xmax: 1067336 ymax: 272811.2
## Projected CRS: NAD83 / New York Long Island (ftUS)
```

The Department of Finance's rolling sales files list tax class 1, 2, and 4 properties that have sold in the last 12-month period in New York City. These files include the neighborhood, building type, square footage, and other data. Rolling sales - last 12 months excluding utility properties, sorted by neighborhood, category.

For practical purpose, and big data limitation, we are only focusing on the May 2024 - April 2025 NYC sales data.

4. Data Cleaning

4.1 Variable Formatting & Renaming

```
all_sales <- bind_rows(
  manhattan,
  queens,
  bronx,
```

```
  brooklyn,
  statenisland
)




all_sales$date_operand <- as.Date(all_sales$SALE.DATE, format = "%m/%d/%Y")

all_sales <- select(all_sales, -SALE.DATE)


all_sales <- all_sales %>% rename(
             borough = BOROUGH,
             neighborhood = NEIGHBORHOOD,
             building_class = BUILDING.CLASS.CATEGORY,
             tax_class = TAX.CLASS.AT.PRESENT,
             block = BLOCK,
             lot = LOT,
             easement = EASEMENT,
             address = ADDRESS,
             apt_numb = APARTMENT.NUMBER,
             zip_code = ZIP.CODE,
             residential_unit = RESIDENTIAL.UNITS,
             commercial_unit = COMMERCIAL.UNITS,
             total_units = TOTAL.UNITS,
             land_sqft = LAND.SQUARE.FEET,
             gross_sqft = GROSS.SQUARE.FEET,
             year_built = YEAR.BUILT,
             tax_class_atsale = TAX.CLASS.AT.TIME.OF.SALE,
             building_class_atsale = BUILDING.CLASS.AT.TIME.OF.SALE,
             sale_price = SALE.PRICE) %>%
             mutate(
    # Replace 1 with "Manhattan", and cast to character
    borough = if_else(borough == 1, "Manhattan", as.character(borough)),
    borough = if_else(borough == 2, "Bronx", as.character(borough)),
    borough = if_else(borough == 3, "Brooklyn", as.character(borough)),
    borough = if_else(borough == 4, "Queens", as.character(borough)),
    borough = if_else(borough == 5, "Staten Island", as.character(borough)),

  )
all_sales <- select(all_sales, -apt_numb, -tax_class_atsale, -building_class_atsale, -BUILDING.CLASS.AT
```

Standardizing our data, making it more readable, and dropping noisy columns.

Variable Definitions

Below are the definitions of variables used in the property sales dataset, based on NYC Department of Finance documentation.

`borough`

The name of the borough where the property is located (e.g., Manhattan, Brooklyn).

`neighborhood`

A commonly recognized name for the area where the property is located, as determined by Department of Finance assessors. Some boundaries and sub-neighborhood designations may vary slightly.

`building_class_category`

A high-level grouping of properties based on general usage (e.g., One-Family Homes, Commercial Condos), useful for filtering and comparison without referencing specific building codes.

`tax_class_at_present`

The property's tax class at the time of data recording. NYC uses four tax classes:

- **Class 1**: Mostly 1–3 unit residential properties, small mixed-use buildings.
- **Class 2**: Larger residential buildings such as condos and co-ops.
- **Class 3**: Utility-owned properties.
- **Class 4**: Commercial and industrial properties (e.g., offices, factories, garages).

`block`

A subdivision of a borough used for real estate tracking, typically representing a side of a street or a city block.

`lot`

A specific parcel of land within a block. Each lot represents a unique property.

`easement`

A legal right to use part of another's property (e.g., transit easement, utility right-of-way).

`building_class_at_present`

A code identifying the structure's use at the time of recording. The first letter denotes the general type (e.g., A = One-Family Home, R = Condo), while the number specifies the style or subtype.

`address`

The full street address of the property. Co-op addresses may include unit or apartment numbers.

`zip_code`

The postal code corresponding to the property's address.

`residential_units`

Number of residential dwelling units on the property.

`commercial_units`

Number of commercial units (e.g., storefronts, offices) on the property.

`total_units`

Total number of units (residential + commercial) on the property.

`land_sqft`

Total square footage of the land parcel.

`gross_sqft`

Total interior floor space, measured from the exterior walls and including all usable floors.

`year_built`

The year the main structure on the property was constructed.

`building_class_at_sale`

The code representing the property's structural use at the time of sale, using the same format as `building_class_at_present`.

`sale_price`

The dollar amount paid by the buyer for the property.

`sale_date`

The official date when the sale was recorded.

`$0_sale_price`

Indicates a property transfer occurred without a monetary transaction (e.g., gift between family members, internal ownership transfer).

4.2 Geocoding & Spatial Join

```r
# Standardize building class
# Building Classification Glossary: https://www.nyc.gov/assets/finance/jump/hlpbldgcode.html
all_sales <- all_sales %>%
  mutate(building_class = recode(building_class,
    "01 ONE FAMILY DWELLINGS" = "1-Family",
    "02 TWO FAMILY DWELLINGS" = "2-Family",
    "03 THREE FAMILY DWELLINGS" = "3-Family",
    "04 TAX CLASS 1 CONDOS" = "Tax Class 1 Condo",
    "07 RENTALS - WALKUP APARTMENTS" = "Walkup Rentals",
    "08 RENTALS - ELEVATOR APARTMENTS" = "Elevator Rentals",
    "09 COOPS - WALKUP APARTMENTS" = "Walkup Coops",
    "10 COOPS - ELEVATOR APARTMENTS" = "Elevator Coops",
    "11 SPECIAL CONDO BILLING LOTS" = "Special Condo Lots",
    "12 CONDOS - WALKUP APARTMENTS" = "Walkup Condos",
    "13 CONDOS - ELEVATOR APARTMENTS" = "Elevator Condos",
    "14 RENTALS - 4-10 UNIT" = "Small Rental (4-10 Units)",
    "15 CONDOS - 2-10 UNIT RESIDENTIAL" = "Small Residential Condo",
    "16 CONDOS - 2-10 UNIT WITH COMMERCIAL UNIT" = "Condo with Commercial",
    "17 CONDO COOPS" = "Condo Coops",
    "21 OFFICE BUILDINGS" = "Offices",
    "22 STORE BUILDINGS" = "Storefronts",
    "25 LUXURY HOTELS" = "Luxury Hotels",
    "26 OTHER HOTELS" = "Other Hotels",
    "27 FACTORIES" = "Factories",
    "28 COMMERCIAL CONDOS" = "Commercial Condos",
    "29 COMMERCIAL GARAGES" = "Commercial Garages",
    "30 WAREHOUSES" = "Warehouses",
    "31 COMMERCIAL VACANT LAND" = "Vacant Commercial Land",
    "32 HOSPITAL AND HEALTH FACILITIES" = "Hospitals/Health",
    "33 EDUCATIONAL FACILITIES" = "Education Facilities",
    "34 THEATRES" = "Theatres",
    "35 INDOOR PUBLIC AND CULTURAL FACILITIES" = "Indoor Cultural Facilities",
    "36 OUTDOOR RECREATIONAL FACILITIES" = "Outdoor Rec Facilities",
    "37 RELIGIOUS FACILITIES" = "Religious Facilities",
    "38 ASYLUMS AND HOMES" = "Asylums/Homes",
    "40 SELECTED GOVERNMENTAL FACILITIES" = "Gov Facilities",
    "41 TAX CLASS 4 - OTHER" = "Tax Class 4 Other",
    "42 CONDO CULTURAL/MEDICAL/EDUCATIONAL/ETC" = "Condo Cultural/Medical",
    "43 CONDO OFFICE BUILDINGS" = "Condo Office",
    "44 CONDO PARKING" = "Condo Parking",
    "45 CONDO HOTELS" = "Condo Hotels",
    "46 CONDO STORE BUILDINGS" = "Condo Storefronts",
    "47 CONDO NON-BUSINESS STORAGE" = "Condo Storage",
    "48 CONDO TERRACES/GARDENS/CABANAS" = "Condo Gardens",
    "49 CONDO WAREHOUSES/FACTORY/INDUS" = "Condo Industrial"
  ))

# Standardize neighboorhood
all_sales <- all_sales %>%
  mutate(neighborhood = recode(neighborhood,
    "ALPHABET CITY" = "Alphabet City",
    "CHELSEA" = "Chelsea",
    "CHINATOWN" = "Chinatown",
```

```
    "CIVIC CENTER" = "Civic Center",
    "CLINTON" = "Clinton",
    "EAST VILLAGE" = "East Village",
    "FASHION" = "Fashion District",
    "FINANCIAL" = "Financial District",
    "FLATIRON" = "Flatiron",
    "GRAMERCY" = "Gramercy",
    "GREENWICH VILLAGE-CENTRAL" = "Greenwich Village (Central)",
    "GREENWICH VILLAGE-WEST" = "Greenwich Village (West)",
    "HARLEM-CENTRAL" = "Harlem (Central)",
    "HARLEM-EAST" = "Harlem (East)",
    "HARLEM-UPPER" = "Harlem (Upper)",
    "HARLEM-WEST" = "Harlem (West)",
    "INWOOD" = "Inwood",
    "JAVITS CENTER" = "Javits Center",
    "KIPS BAY" = "Kips Bay",
    "LITTLE ITALY" = "Little Italy",
    "LOWER EAST SIDE" = "Lower East Side",
    "MANHATTAN VALLEY" = "Manhattan Valley",
    "MIDTOWN CBD" = "Midtown CBD",
    "MIDTOWN EAST" = "Midtown East",
    "MIDTOWN WEST" = "Midtown West",
    "MORNINGSIDE HEIGHTS" = "Morningside Heights",
    "MURRAY HILL" = "Murray Hill",
    "ROOSEVELT ISLAND" = "Roosevelt Island",
    "SOHO" = "SoHo",
    "SOUTHBRIDGE" = "Southbridge",
    "TRIBECA" = "TriBeCa",
    "UPPER EAST SIDE (59-79)" = "Upper East Side (59-79)",
    "UPPER EAST SIDE (79-96)" = "Upper East Side (79-96)",
    "UPPER EAST SIDE (96-110)" = "Upper East Side (96-110)",
    "UPPER WEST SIDE (59-79)" = "Upper West Side (59-79)",
    "UPPER WEST SIDE (79-96)" = "Upper West Side (79-96)",
    "UPPER WEST SIDE (96-116)" = "Upper West Side (96-116)",
    "WASHINGTON HEIGHTS LOWER" = "Washington Heights (Lower)",
    "WASHINGTON HEIGHTS UPPER" = "Washington Heights (Upper)"
  ))
```

Sample use of Census API in the case that our observations are below 10000, and for future expansion on regions outside of NYC

```
library(tidygeocoder)
library(purrr)

# Source: https://stackoverflow.com/questions/68439563/get-latitude-and-longitude-in-r-using-address
# Create a duplicate object to avoid overwriting
# API Key: KEYHERE
options(census_api_key = "KEYHERE")
```

4.2 Geocoding & Spatial Join

```
# Documentation on how to calculate PLUTO's Borough, Tax Block & Lot (BBL)
# https://www.nyc.gov/assets/planning/download/pdf/data-maps/open-data/pluto_datadictionary.pdf

pluto <- pluto %>%
  select(BBL,BoroCode, geometry, Borough) %>%
  st_make_valid()%>%
  st_transform(4326)%>%
  mutate(
    cent = st_centroid(geometry),
    long = st_coordinates(st_centroid(cent))[,1],
    lat = st_coordinates(st_centroid(cent))[,2]
  ) %>%
  select(-geometry, -cent)

# Adding geo identifiers
addy <- all_sales
addy <- addy %>%
  rename(
    street = address,
    postalcode = zip_code
  ) %>%
    mutate(
    state   = "NY",
    country = "US"
  )


sales_geo <- addy %>%
  mutate(
    borough_code = case_when(
      borough == "Manhattan"     ~ 1L,
      borough == "Bronx"         ~ 2L,
        borough == "Brooklyn"      ~ 3L,
        borough == "Queens"        ~ 4L,
        borough == "Staten Island" ~ 5L,
        TRUE                       ~ NA_integer_
      ),
    BBL = sprintf(
      "%1d%05d%04d",
      borough_code,   # 1 digit
      block,          # zero-pad to width 5
      lot             # zero-pad to width 4
    ) %>% as.numeric()
  )

sales_geo <- sales_geo %>%
  left_join(pluto, by = c("BBL", "borough_code" = "BoroCode"))
```

In this chunk we use NYC's PLUTO shapefile to attach latitude/longitude coordinates to each sales record. First, we read in the MapPLUTO polygons and select only the Borough-Block-Lot (BBL) code, borough code, and geometry. We then repair any invalid geometries, reproject to WGS84 (EPSG:4326), compute

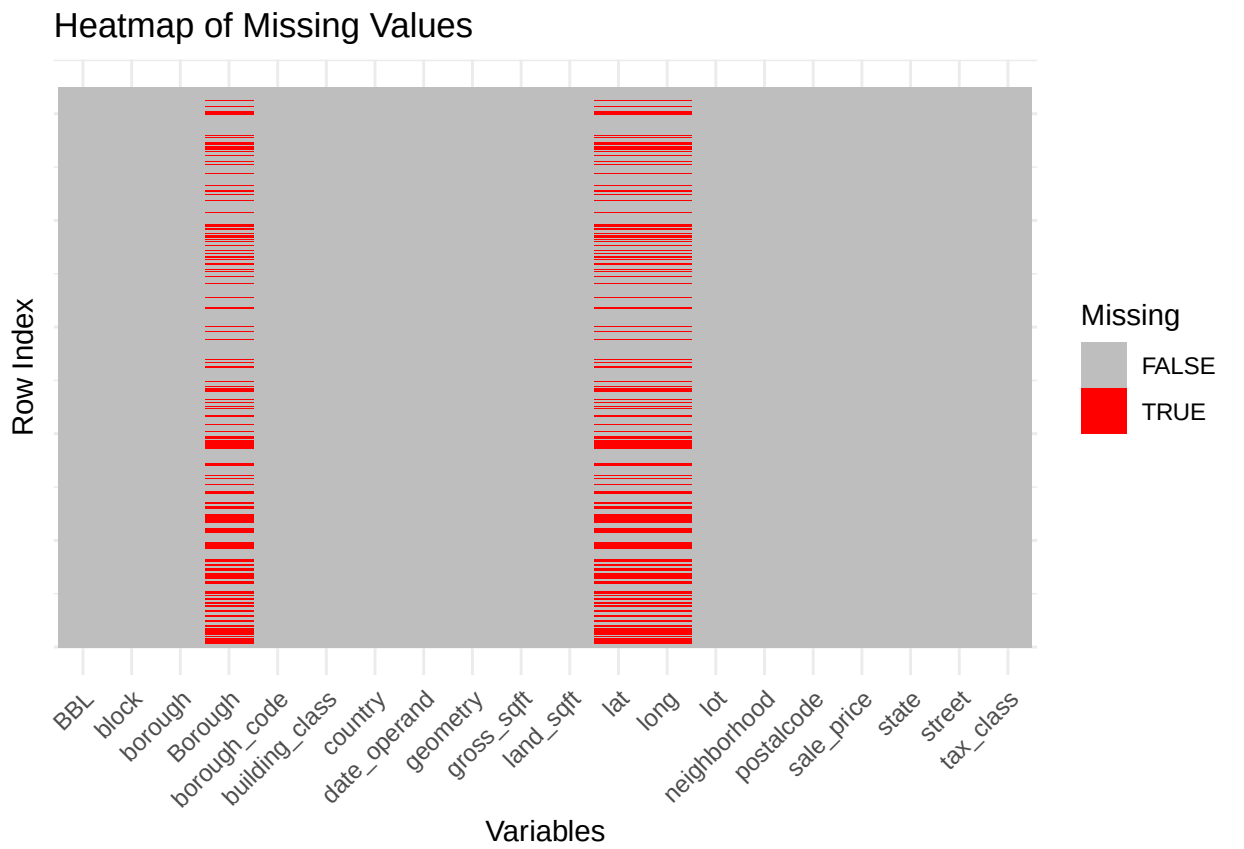
each lot's centroid, and extract its X/Y coordinates. Next, we build the matching 10-digit BBL in our `addy` sales table (by zero-padding block and lot and using the borough code), and perform a left join on `BBL` and `borough_code = BoroCode` so that every sale picks up its corresponding `long` and `lat`. The result is a `sales_geo` data frame that is fully geocoded for mapping and spatial analysis.

4.3 Missing Values Exploration with Heatmap

```
# Keep only true cash sales (>0)
total_zero <- sum(sales_geo$sale_price == 0)
sales_geo <- sales_geo %>%
  mutate(across(everything(), as.character)) %>%
    filter(!is.na(sale_price), sale_price > 0)

## Utilizing ggplot2 to perform the same method
missing_val <- sales_geo %>%
  mutate(row_id = row_number()) %>%
  pivot_longer(-row_id, names_to = "variable", values_to = "value") %>%
  mutate(Missing = is.na(value) | value == "NA")

ggplot(missing_val, aes(x = variable, y = row_id, fill = Missing)) +
  geom_tile() +
  scale_fill_manual(values = c("FALSE" = "grey", "TRUE" = "red")) +
  theme_minimal() +
  labs(title = "Heatmap of Missing Values", x = "Variables", y = "Row Index") +
  theme(axis.text.y = element_blank(),
        axis.ticks.y = element_blank(),
        axis.text.x = element_text(angle = 45, hjust = 1)#helps make variables readable
        )
```



Seems like we are missing a lot data. Let's investigate on why this is the case before we remove them. Though, there are as much observations as missing values in the easement column. Hence, we can drop the column.

Four key missing variables

`Commercial Unit`

Missing values in the commercial_unit column likely occur when a property is entirely residential, such as a single-family home, co-op, or condo unit. In such cases, the number of commercial units would be 0, but may be left as NA if the data system omits zero-filled values for non-commercial properties. Additionally, tax lots that represent vacant land or easements may not have any unit structure defined.

`Residential Unit`

Similar to commercial_unit, residential_unit may be missing for properties that are fully commercial — such as warehouses, office buildings, or parking lots — where no residential dwellings exist. Certain building types (e.g., utility structures or government buildings) may not fall under traditional housing classifications, leading to unrecorded residential unit counts. ### `Total Units` total_units might be derived from summing residential_unit and commercial_unit. If either of those components is missing, total_units may be left blank as well. Additionally, for some institutional, mixed-use, or vacant land properties, unit data may not be clearly defined or reported during sale transactions.

Year Built

The year_built field is often missing for: - Vacant land where no building has been constructed yet. - Old or irregular buildings where historical records are incomplete or unclear. - Co-op apartments or condo units, especially if the sale record reflects the unit and not the parent building, in which case building-level metadata like year_built may not be linked.

There are 25191 $0 transactions through these boroughs in NYC. These $0 transactions can be explained by the following context and content description for the dataset:

- Many sales occur with a nonsensically small dollar amount: $0 most commonly. These sales are actually transfers of deeds between parties: for example, parents transferring ownership to their home to a child after moving out for retirement.
- This dataset uses the financial definition of a building/building unit, for tax purposes. In case a single entity owns the building in question, a sale covers the value of the entire building. In case a building is owned piecemeal by its residents (a condominium), a sale refers to a single apartment (or group of apartments) owned by some individual.

Since the missing values can be explained by the above context, we will drop the missing values from the data.

5. Exploratory Visualizations

5.1 Waffle Chart of Residential Transactions

```
library(waffle)

# We want to focus on residential building - tax class = 1 or 2
borough_counts <- sales_geo %>%
  filter(!is.na(borough), !is.na(building_class)) %>%
  group_by(borough, building_class) %>%
  summarise(transactions = n()) %>%
  arrange(borough, desc(transactions))
```

```
## `summarise()` has grouped output by 'borough'. You can override using the
## `.groups` argument.
```

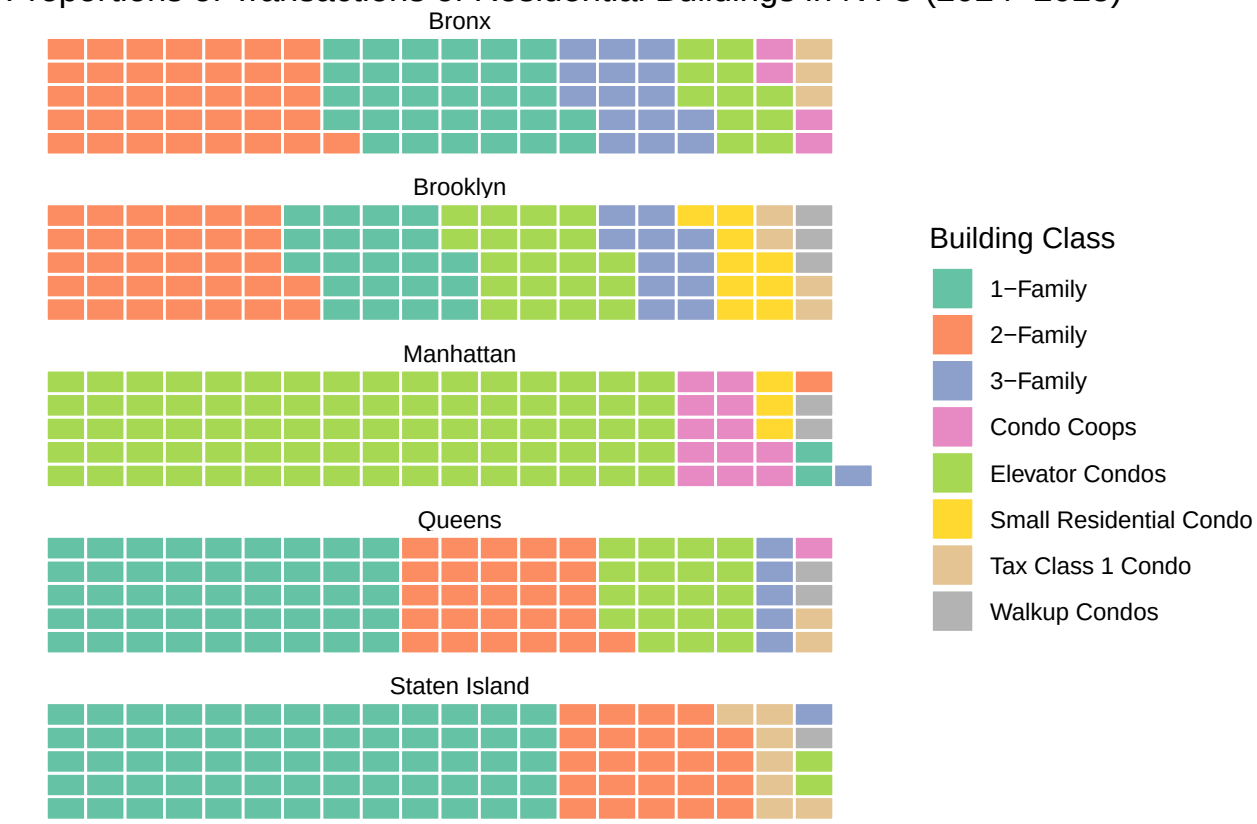
```
# We would like to filter out residential buildings
residential_classes <- c(
  "1-Family",
  "2-Family",
  "3-Family",
  "Tax Class 1 Condo",
  "Walkup Condos",
  "Elevator Condos",
  "Small Residential Condo",
  "Condo Coops"
)
residential_borough_counts <- borough_counts %>%
  filter(building_class %in% residential_classes)
```

```r
# Calculate the proportions of Building purchased for each borough
residential_borough_counts <- residential_borough_counts %>%
  group_by(borough) %>%
  mutate(
    proportion = transactions / sum(transactions),
    waffle_prop = round(proportion*100)
  )

head(residential_borough_counts)
```

```
## # A tibble: 6 x 5
## # Groups:   borough [1]
##   borough building_class    transactions proportion waffle_prop
##   <chr>   <chr>                    <int>      <dbl>       <dbl>
## 1 Bronx   2-Family                   963     0.365           36
## 2 Bronx   1-Family                   809     0.307           31
## 3 Bronx   3-Family                   387     0.147           15
## 4 Bronx   Elevator Condos             290     0.110           11
## 5 Bronx   Condo Coops                 99     0.0375           4
## 6 Bronx   Tax Class 1 Condo           79     0.0299           3
```

```r
# special icons for viz
# 100 squares per waffle, so we'll scale to hundreds
ggplot(residential_borough_counts,
       aes(values = waffle_prop, fill = building_class)) +
  geom_waffle(
    n_rows = 5,
    glyph  = "home",
    size   = 0.5,
    color  = "white"
  ) +
  facet_wrap(~borough, ncol = 1)  +
  scale_fill_brewer("Building Class", palette = "Set2") +
  labs(
    title = "Proportions of Transactions of Residential Buildings in NYC (2024-2025)") + theme_void() +
  theme(
    legend.position = "right",
    strip.text.y    = element_text(angle = 0))
```




```
    plot.title = element_text(size = 25, hjust = 1, face="bold", color = "#4e4d47")
```

5.2 Time-series Average Price Heatmap

```
library(viridis)
```

```
## Loading required package: viridisLite
```

```
library(ggExtra)

heat_avg <- all_sales %>%
  filter(!is.na(sale_price)) %>%
  filter(sale_price > 0) %>%
  mutate(month_year = floor_date(date_operand, "month"),
         sale_price = as.numeric( gsub(",", "", sale_price))/1e6 #strip out commas before converting t
) %>%
  group_by(month_year,borough) %>%
  summarise(
    avg_price = mean(sale_price),
    .group="drop")
```

```
## `summarise()` has grouped output by 'month_year'. You can override using the
## `.groups` argument.
```

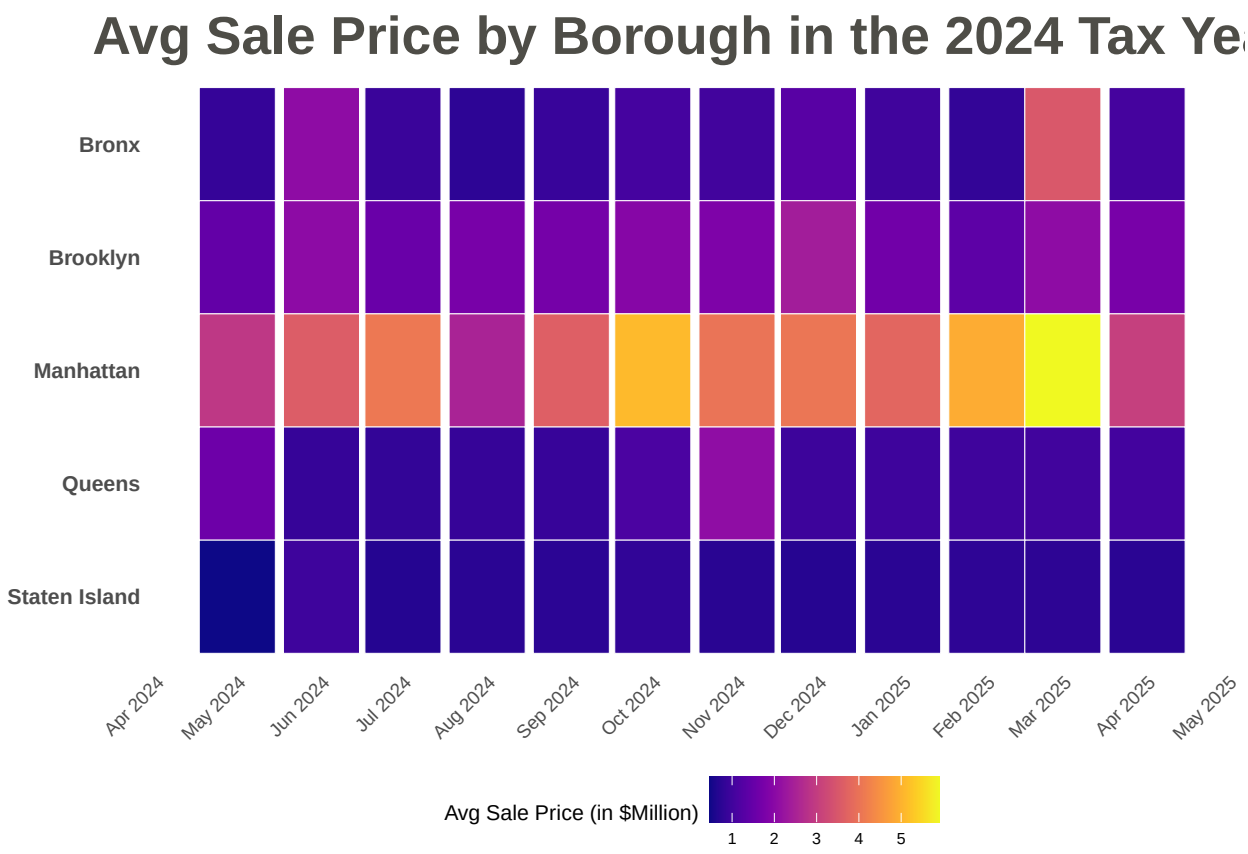
```r
head(heat_avg)
```

```
## # A tibble: 6 x 4
## # Groups:   month_year [2]
##   month_year borough       avg_price .group
##   <date>     <chr>             <dbl> <chr>
## 1 2024-05-01 Bronx             0.857 drop
## 2 2024-05-01 Brooklyn          1.46  drop
## 3 2024-05-01 Manhattan         2.91  drop
## 4 2024-05-01 Queens            1.59  drop
## 5 2024-05-01 Staten Island     0.473 drop
## 6 2024-06-01 Bronx             2.08  drop
```

```r
  #filter(!is.na(sale_price), sale_price > 0)

ggplot(heat_avg, aes(x = month_year, y = borough, fill = avg_price)) +
  geom_tile(color = "white", size = 0.1) +
  scale_fill_viridis(name = "Avg Sale Price (in $Million)", option = "C") +
  # borough on y is categorical, so use scale_y_discrete()
  scale_y_discrete(limits = rev(unique(heat_avg$borough))) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b %Y") +
  theme_minimal(base_size = 8) +
  labs(
    title = "Avg Sale Price by Borough in the 2024 Tax Year",
    y= NULL,
    x=NULL
  ) +
  theme(
    axis.text.x      = element_text(angle = 45, hjust = 1, size = 7),
    axis.text.y      = element_text(size = 8, face="bold"),
    legend.position  = "bottom",
    plot.title = element_text(size = 20, hjust = 0.5, face="bold", color = "#4e4d47"),
    strip.background = element_rect(fill = "white"),
    legend.title     = element_text(size = 8),
    legend.text      = element_text(size = 6),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    legend
  )
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



Spatial Maps

```
library(giscoR)

usa_map <- gisco_get_countries(year = "2024", country = "US", resolution = 1) %>%
  st_transform(usa_map, crs = 4326)


#Filter the NAs points in terms of long and lat
map_points <- sales_geo %>%
  filter(!is.na(long), !is.na(lat)) %>%
  mutate(long = as.numeric(long),
         lat = as.numeric(lat),
          sale_price = as.numeric( gsub(",", "", sale_price)) / 100000 #strip out commas before convert
         ) %>%
  arrange(sale_price)
# Color Scale breaks
mybreaks <- c(0.02, 0.04, 0.08, 1, 7)

borough_pins <- tibble(
  BoroName = c("Manhattan", "Bronx",     "Brooklyn",   "Queens",       "Staten Island"),
  lon      = c(-73.97,      -73.90,      -73.94,       -73.80,         -74.15),
  lat      = c( 40.78,       40.85,       40.65,        40.72,          40.58 )
)
```

5.3 Property Sales Bubble Map

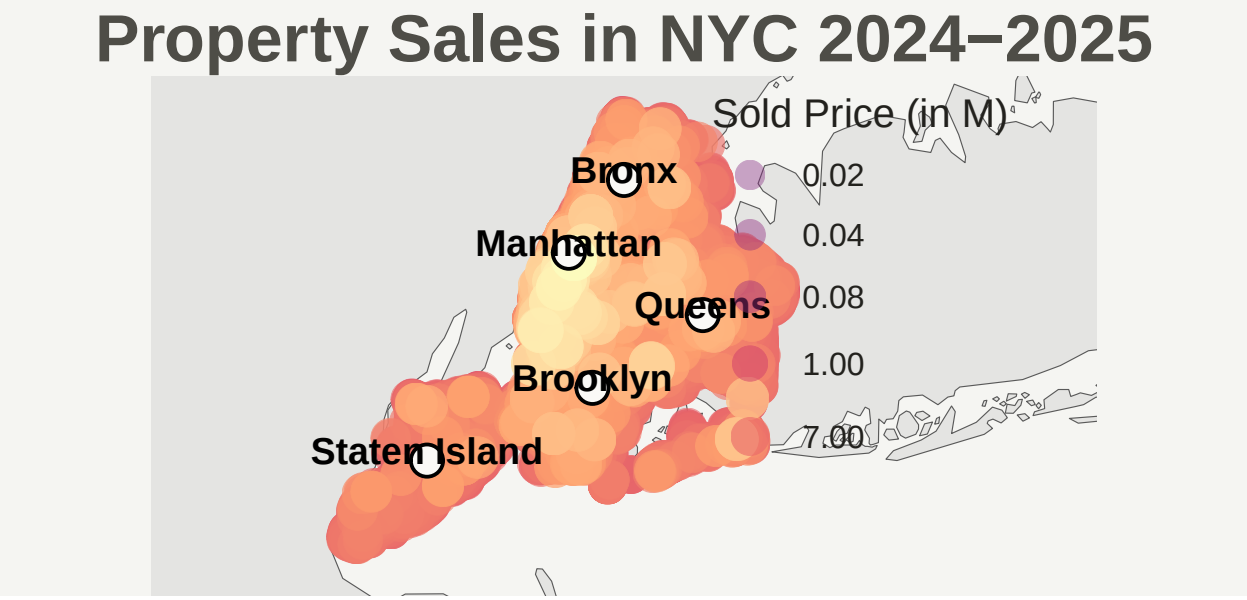
```
ggplot() +
# a) USA basemap
geom_sf(data = usa_map, fill= "grey", alpha = 0.3) +
# b) Points for each sale
geom_point(data=map_points, aes(x = long, y = lat, color= sale_price, size = sale_price, alpha = sale
  shape = 20, stroke = FALSE
  ) +
scale_size_continuous(
  name = "Sold Price (in M)", trans = "log",
  range = c(1, 12), breaks = mybreaks
) +
scale_alpha_continuous(
  name = "Sold Price (in M)", trans = "log",
  range = c(0.1, .9), breaks = mybreaks
) +
scale_color_viridis_c(
  option = "magma", trans = "log",
  breaks = mybreaks, name = "Sold Price (in M)"
) +
  # borough pins
  geom_point(
  data  = borough_pins,
  aes(x = lon, y = lat),
  shape = 21, fill = "#FAF9F6", color = "black", size = 5, stroke = 1
) +
geom_text(
  data = borough_pins,
  aes(x = lon, y = lat, label = BoroName),
  nudge_y = 0.01, size = 5, fontface = "bold"
) +
  # zoom to nyc
coord_sf(
  xlim   = c(-74.5, -73.3),
  ylim   = c(40.45, 40.95),
  expand = FALSE
) +
# c) Zoom in on NYC
coord_sf(xlim = c(-74.5, -73.3), ylim = c(40.45,  40.95), expand = FALSE) +
# d) Labels and theme
guides(color = guide_legend()) +
# Theme Goal: no grid, clean, light background
theme_void() +
theme(
  text = element_text(color = "#22211d"),
  plot.margin = margin(r = 2, l = 2, unit = "cm"),
  plot.background = element_rect(fill = "#f5f5f2", color = NA),
  panel.background = element_rect(fill = "#f5f5f2", color = NA),
  plot.title = element_text(size = 25, hjust = 0.5, face="bold", color = "#4e4d47"),
  legend.position = c(0.75, 0.6),
  legend.title = element_text(size = 15),
  legend.text = element_text(size = 12),
```

```
    legend.key        = element_blank()

  ) +
  labs(
    title ="Property Sales in NYC 2024-2025"
  )
```

```
## Coordinate system already present. Adding new coordinate system, which will
## replace the existing one.
```

```
## Warning: A numeric ‘legend.position‘ argument in ‘theme()‘ was deprecated in ggplot2
## 3.5.0.
## i Please use the ‘legend.position.inside‘ argument of ‘theme()‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



5.4 Property Sales Density Map

```
library(RColorBrewer)
head(map_points)
```

```
##     borough                neighborhood            building_class tax_class
## 1 Manhattan Greenwich Village (Central)            Elevator Coops         2
## 2 Manhattan    Greenwich Village (West) Small Rental (4-10 Units)        2A
```

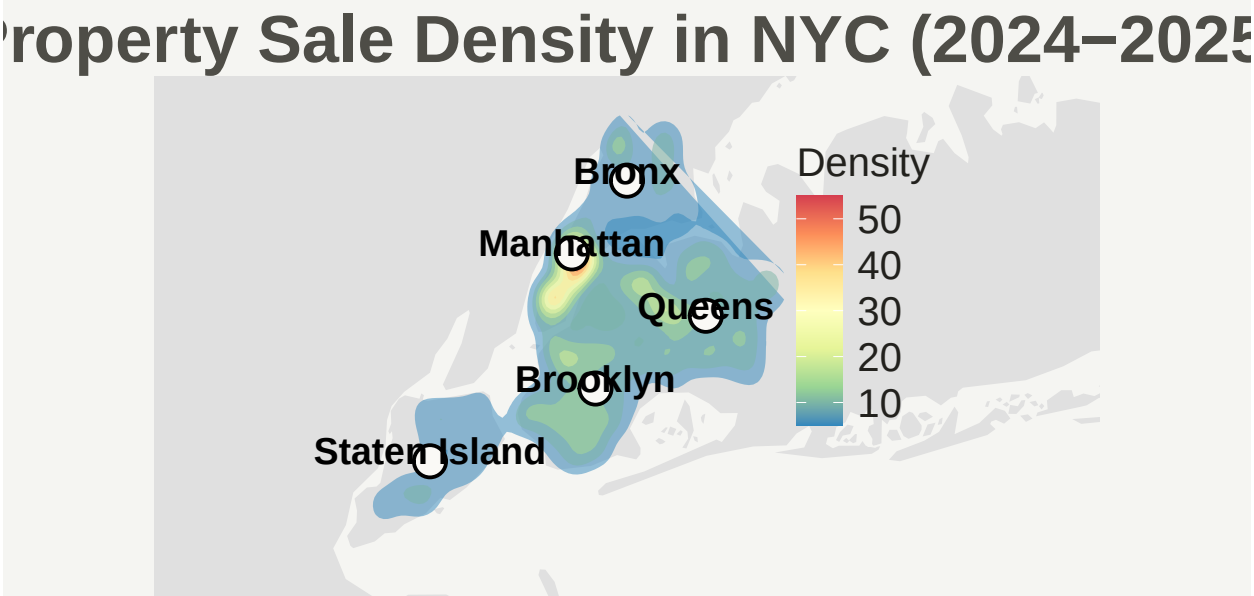
```
## 3 Manhattan    Greenwich Village (West)               Storefronts         4
## 4 Manhattan            Harlem (Central)            Walkup Rentals        2A
## 5 Manhattan               Harlem (East)             Asylums/Homes         4
## 6 Manhattan                Midtown East            Elevator Coops         2
##   block lot                    street postalcode land_sqft gross_sqft
## 1   573  43      24 FIFTH AVENUE, 610      10011
## 2   618   9        248 WEST 14 STREET      10011     2,478      8,400
## 3   618  10        244 WEST 14 STREET      10011     5,163     10,300
## 4  1903  24               111 W 118TH      10026     2,018      3,750
## 5  1667  12     223 EAST 117TH STREET      10035    10,092     34,050
## 6  1333  42 314 EAST 41ST STREET, 105      10017
##   sale_price date_operand state country borough_code        BBL Borough
## 1      1e-05   2025-03-15    NY      US            1 1005730043      MN
## 2      1e-05   2025-01-08    NY      US            1 1006180009      MN
## 3      1e-05   2025-01-08    NY      US            1 1006180010      MN
## 4      1e-05   2024-10-10    NY      US            1 1019030024      MN
## 5      1e-05   2024-12-19    NY      US            1 1016670012      MN
## 6      1e-05   2024-09-16    NY      US            1 1013330042      MN
##        long      lat
## 1 -73.99631 40.73325
## 2 -74.00211 40.73927
## 3 -74.00199 40.73922
## 4 -73.94944 40.80388
## 5 -73.93820 40.79831
## 6 -73.97210 40.74868
##
## 1
## 2
## 3
## 4
## 5 list(c(-73.9379511794782, -73.9381354058399, -73.938216379383, -73.9382966126352, -73.938377714766
## 6
```

```
  ggplot() +
  #Base map of USA
  geom_sf(data = usa_map, fill="grey88", color = NA)+
  # 2D density heat
  stat_density_2d(data= map_points, aes(x=long, y=lat, fill = ..level..), geom="polygon", alpha=0.5, co
  scale_fill_gradientn(colours = rev(brewer.pal(7,"Spectral")),
                       name = "Density")+
  # borough pins
    geom_point(
    data  = borough_pins,
    aes(x = lon, y = lat),
    shape = 21, fill = "#FAF9F6", color = "black", size = 5, stroke = 1
  ) +
  geom_text(
    data = borough_pins,
    aes(x = lon, y = lat, label = BoroName),
    nudge_y = 0.01, size = 5, fontface = "bold"
  ) +
    # zoom to nyc
  coord_sf(
    xlim   = c(-74.5, -73.3),
```

```
    ylim   = c(40.45, 40.95),
    expand = FALSE
  ) +

  theme_void() +
  theme(
    text = element_text(color = "#22211d"),
    plot.margin = margin(r = 2, l = 2, unit = "cm"),
    plot.background = element_rect(fill = "#f5f5f2", color = NA),
    panel.background = element_rect(fill = "#f5f5f2", color = NA),
    plot.title = element_text(size = 25, hjust = 0.5, face="bold", color = "#4e4d47"),
    legend.position = c(0.75, 0.6),
    legend.title = element_text(size = 15),
    legend.text = element_text(size = 15),
    legend.key        = element_blank()
  ) +
    labs(title = "Property Sale Density in NYC (2024-2025)")
```

```
## Warning: The dot-dot notation (‘..level..‘) was deprecated in ggplot2 3.4.0.
## i Please use ‘after_stat(level)‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



References

1. NYC Department of Finance, Rolling Sales Data, NYC rolling-sales CSVs. https://www.nyc.gov/site/finance/property/property-rolling-sales-data.page
2. NYC Planning, PLUTO Data Dictionary. https://www.nyc.gov/assets/planning/download/pdf/data-maps/open-data/pluto_datadictionary.pdf
3. StackOverflow user "saurav", Get latitude and longitude in R using address (tidygeocoder). https://stackoverflow.com/questions/68439563/get-latitude-and-longitude-in-r-using-address
4. Tidygeocoder package, tidygeocoder. https://jessecambon.github.io/tidygeocoder/